SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA3714**

Course Name: **SOFTWARE TESTING**

Duration: **1 Week**

Submission Date: 6-11-2026

Date of Issue: 3-11-2026 Max Marks: **50 Marks**

ASSIGNMENT – 2

CO	Course Outcome Statement
CO2	Design effective test cases using black-box, white-box testing techniques and to control methods flow.

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	**Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.**

S2	**Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.**
S3	**Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.**
S4	**Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.**

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2    return INVALID
 3  rate = 40
 4  if zone == "B" then
 5    rate = 60
 6  else if zone == "C" then
 7    rate = 90
 8  charge = 0
 9  w = weight
10  while w > 0 do
11      charge = charge + rate
12    w = w - 1
13  if express then
14    if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
```

```
18      charge = charge * 0.9

19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

TASK 2 – Evaluate Random Testing 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion.

Evidence Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**
- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)**
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

TASK 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**
- **Tests for the "maximum two retries" rule and the cancellation rule**

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

TASK 7 – Construct the Control Flow Graph 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**

- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**
- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**
- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**
- **Why this suite is better than the alternatives**
- **The coverage and the rule combinations it achieves**
- **The number of test cases and the effort needed to run them**
- **Known gaps and risks that remain**

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**
- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected

A clear analysis of how the changes affect the original test design.

B. Evaluation of the Original Suite – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**

- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

- **Describe the new and changed test cases**
- **Explain which techniques you used to derive them**
- **Show the effect on coverage and the number of test cases**

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

D. Evidence-Based Defence – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Use:

- **Equivalence classes and boundaries**
- **Decision-table combinations**
- **Cyclomatic complexity and basis paths**

- **Coverage achieved**
- **Execution time limit**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Select Test Case Design Strategies	**5**

Task 2 – Evaluate Random Testing	**2**
Task 3 – Design and Evaluate Equivalence Classes	**6**
Task 4 – Design and Evaluate Boundary Value Tests	**4**
Task 5 – Apply Other Black-Box Techniques	**3**
Task 6 – Design and Evaluate State-Transition Tests	**3**
Task 7 – Construct the Control Flow Graph	**4**
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	**4**
Task 9 – Judge Test Adequacy and Coverage	**3**
Task 10 – Compare Black-Box and White-Box Effectiveness	**4**
Task 11 – Recommend and Defend the Final Test Suite	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

CSA37 – SOFTWARE TESTING

ASSIGNMENT – 2

TASK 1: SELECT TEST CASE DESIGN STRATEGIES

Software testing techniques are used to identify defects and verify whether a software application satisfies its requirements. For the SwiftShip shipping-charge calculator, both black-box and white-box testing techniques are required because the application contains business rules, calculations, conditions, loops and tracking-state transitions.

1. Black-Box Testing

Black-box testing verifies the functionality of the application without examining its internal source code. Test cases are designed using the requirements and expected outputs.

The following techniques are suitable:

- **Equivalence Class Partitioning:** Divides input values into valid and invalid groups. For example, valid weights are greater than 0 kg and up to 50 kg.
- **Boundary Value Analysis:** Tests values at and around important limits, such as 0, 0.1, 30, 30.1, 50 and 50.1 kg.
- **Decision Table Testing:** Tests combinations of express delivery, discount eligibility and other business conditions.
- **State-Transition Testing:** Verifies whether parcel tracking follows the permitted sequence of states.

2. White-Box Testing

White-box testing examines the internal program structure, logic and execution paths.

The following techniques are suitable:

- **Statement Coverage:** Ensures that every executable statement is executed at least once.
- **Branch Coverage:** Checks both true and false outcomes of decision statements.
- **Condition Coverage:** Verifies individual conditions within compound decisions.
- **Path Testing:** Executes independent paths through the program.

- **Loop Testing:** Checks the behaviour of the weight calculation loop for different numbers of iterations.

3. Information Required

For black-box testing, the tester requires the functional specification, input constraints, expected outputs, discount rules and parcel tracking rules.

For white-box testing, the tester requires the source code or pseudocode, control flow, decision statements, loops and internal program structure.

4. Defects Identified by Each Technique

Black-box testing can identify incorrect shipping charges, invalid input acceptance, incorrect discounts, express delivery violations and invalid parcel-state transitions.

White-box testing can identify missing branches, incorrect loop behaviour, unreachable statements, logical errors and insufficient execution-path coverage.

5. Recommended Strategy

A combination of black-box and white-box testing is recommended. Black-box testing verifies whether the application meets customer and business requirements, while white-box testing verifies whether the internal implementation works correctly.

Using both approaches provides stronger test coverage and reduces the possibility of defects reaching production.

TASK 2: EVALUATE RANDOM TESTING

Random testing generates input values automatically or selects them randomly from the permitted input domain. For SwiftShip, 1,000 random test cases can be generated using different weights, delivery zones, express options and monthly parcel counts.

1. How Random Testing Can Be Performed

The tester generates random combinations of:

- Weight between valid and invalid values.
- Zone A, Zone B, Zone C and invalid zones.
- Express delivery enabled or disabled.
- Parcel counts below and above 10.

Each generated input is submitted to the application, and the actual output is compared with the expected output calculated from the requirements.

2. Probability of Testing Important Boundary Values

If the generator selects uniformly from the 500 valid one-decimal weight values between 0.1 kg and 50.0 kg, the probability of selecting one particular value is:

Probability = 1 / 500 = 0.002 = 0.2%

For 1,000 random tests, the expected number of occurrences of a particular value is:

Expected occurrences = 1,000 × 1/500 = 2

Therefore, values such as 30.0 kg, 30.1 kg and 50.0 kg may be selected, but random testing does not guarantee that every important boundary will be tested.

The actual probability also depends on how the random generator selects values and whether it includes invalid inputs.

3. Limitations of Random Testing

- Important boundary values may be missed.
- Rare input combinations may not occur.
- Random testing does not guarantee complete branch or path coverage.
- Expected results must still be independently calculated.
- Reproducing a failure may be difficult unless the random seed and generated inputs are recorded.

4. Evaluation

Random testing is useful for discovering unexpected input combinations and unexpected failures. However, it should not be the primary testing technique for SwiftShip because incorrect charges can cause financial loss and customer complaints.

Boundary Value Analysis, Equivalence Class Partitioning and Decision Table Testing should be used as the main techniques. Random testing can be added as supplementary testing.

TASK 3: DESIGN AND EVALUATE EQUIVALENCE CLASSES

Equivalence Class Partitioning (ECP) divides the input domain into groups in which the application is expected to behave similarly. Instead of testing every possible input, representative values are selected from each group.

For SwiftShip, the four major input parameters are weight, delivery zone, express option and monthly parcel count.

1. Equivalence Classes

A. Weight

Class	Input Range	Expected Result
Valid	Greater than 0 and up to 50 kg, with at most one decimal place	Accepted
Invalid	0 or negative weight	INVALID
Invalid	Greater than 50 kg	INVALID
Invalid	More than one decimal place	INVALID

B. Delivery Zone

Class	Input	Expected Result
Valid	Zone A	₹40 per started kg
Valid	Zone B	₹60 per started kg
Valid	Zone C	₹90 per started kg
Invalid	Any other zone	INVALID

C. Express Delivery

Class	Input	Expected Result
Valid	Express disabled	Normal charge
Valid	Express enabled and weight up to 30 kg	Additional 50%
Restricted	Express enabled and weight above 30 kg	NOT_ALLOWED

D. Monthly Parcel Count

Class	Input	Expected Result
Valid	Fewer than 10 parcels	No discount
Valid	10 or more parcels	10% discount

2. Representative Test Cases

Test ID	Weight	Zone	Express	Parcels	Expected Output
TC01	1.0	A	No	9	₹40
TC02	1.1	B	No	9	₹120
TC03	2.0	C	Yes	9	₹270
TC04	1.0	A	No	10	₹36
TC05	1.0	B	Yes	10	₹81
TC06	30.0	A	Yes	9	₹1,800
TC07	30.1	A	Yes	9	NOT_ALLOWED
TC08	50.0	C	No	9	₹4,500
TC09	0	A	No	9	INVALID
TC10	50.1	A	No	9	INVALID
TC11	1.0	X	No	9	INVALID
TC12	1.25	A	No	9	INVALID

3. Calculation of Expected Results

For TC02:

Weight = 1.1 kg
Started kilograms = 2
Zone B rate = ₹60

Charge = 2 × 60 = ₹120

For TC03:

Weight = 2 kg
Zone C rate = ₹90
Express charge = 50% extra

Charge = 2 × 90 × 1.5 = ₹270

For TC05:

Weight = 1 kg
Zone B rate = ₹60
Express multiplier = 1.5
Discount multiplier = 0.9

Final charge = 60 × 1.5 × 0.9 = ₹81

4. Evaluation of the Pseudocode

The pseudocode correctly handles invalid weight limits, started-kilogram calculation, express restrictions and the discount threshold.

However, it does not explicitly validate invalid delivery zones. An invalid zone such as X receives the default Zone A rate of ₹40 instead of returning INVALID.

It also does not check whether the weight contains more than one decimal place. For example, 1.25 kg is processed by the loop and produces a charge for two started kilograms.

Therefore, TC11 and TC12 are expected to reveal defects in the current pseudocode.

Conclusion: Equivalence Class Partitioning reduces the number of test cases while ensuring that all important valid and invalid input groups are tested.

TASK 4: DESIGN AND EVALUATE BOUNDARY VALUE TESTS

Boundary Value Analysis (BVA) focuses on values at the limits of input ranges because defects frequently occur near these limits.

For SwiftShip, important boundaries exist in weight, express delivery eligibility and monthly parcel count.

1. Important Boundary Values

Requirement	Boundary Values	Purpose
Minimum weight	0 and 0.1 kg	Verify minimum valid weight
Express limit	30 and 30.1 kg	Verify express eligibility
Maximum weight	50 and 50.1 kg	Verify maximum valid weight
Discount threshold	9 and 10 parcels	Verify discount eligibility

The started-kilogram rule also requires testing values around whole kilograms. For example, 1.0 kg requires one started kilogram, whereas 1.1 kg requires two.

2. Weight Boundary Test Cases

Test ID	Weight	Zone	Express	Expected Result
BVA01	0	A	No	INVALID
BVA02	0.1	A	No	₹40
BVA03	1.0	A	No	₹40
BVA04	1.1	A	No	₹80
BVA05	30.0	A	Yes	₹1,800
BVA06	30.1	A	Yes	NOT_ALLOWED
BVA07	50.0	C	No	₹4,500
BVA08	50.1	C	No	INVALID

3. Parcel Discount Boundary Test Cases

Test ID	Parcels	Weight	Zone	Expected Result
BVA09	9	1.0	A	₹40
BVA10	10	1.0	A	₹36

At 9 parcels, the customer does not receive a discount. At 10 parcels, the 10% discount is applied.

4. Types of Boundary Value Analysis

Normal BVA: Tests the minimum, maximum and nominal values of each numerical input. For two numerical inputs (weight and parcel count), the common formula is:

Normal BVA = 4n + 1
= 4(2) + 1
= 9 test cases.

Robust BVA: Includes values immediately outside the valid boundaries in addition to the normal boundary values.

Robust BVA = 6n + 1
= 6(2) + 1
= 13 test cases.

Worst-Case BVA: Tests all combinations of the selected five boundary values for each numerical input.

Worst-Case BVA = 5^n
= 5^2
= 25 test cases.

These counts apply to the two numerical dimensions. Delivery zone is categorical and express is Boolean, so they should be covered using equivalence classes and decision tables rather than numerical BVA.

5. Recommendation

Robust BVA is recommended because it tests both valid and invalid values near the limits. It is particularly important for SwiftShip because a small boundary error can cause incorrect charges or allow restricted express deliveries.

TASK 5: APPLY OTHER BLACK-BOX TESTING TECHNIQUES

Decision Table Testing is suitable for SwiftShip because the final shipping charge depends on multiple business conditions.

The important conditions are express delivery, express weight eligibility and discount eligibility.

1. Decision Table

Conditions / Rules	R1	R2	R3	R4	R5
Express selected?	No	Yes	Yes	No	Yes
Weight above 30 kg?	-	No	Yes	-	No
Parcels 10 or more?	No	No	No	Yes	Yes
Normal charge	Yes	Yes	-	Yes	Yes
Add 50% express charge	No	Yes	-	No	Yes
Return NOT_ALLOWED	No	No	Yes	No	No
Apply 10% discount	No	No	-	Yes	Yes

Here, "-" means that the condition does not affect the outcome for that rule.

2. Derived Test Cases

Test ID	Weight	Express	Parcels	Expected Result (Zone A)
DT01	1.0	No	9	₹40
DT02	1.0	Yes	9	₹60
DT03	31.0	Yes	9	NOT_ALLOWED
DT04	1.0	No	10	₹36
DT05	1.0	Yes	10	₹54

3. Why Decision Table Testing Is Useful

Equivalence Class Partitioning checks representative values from input groups, while Boundary Value Analysis focuses on limits. However, these techniques alone may not test every interaction between express delivery and discount eligibility.

Decision Table Testing verifies combinations of conditions and ensures that the correct action is performed for each combination.

4. Evaluation

Decision Table Testing is effective for financial calculations because it helps identify missing or incorrect business rules. It should be combined with ECP and BVA to provide stronger functional coverage.

TASK 6: DESIGN STATE-TRANSITION TESTS

State-Transition Testing verifies how a parcel moves from one tracking state to another. SwiftShip uses a defined sequence of delivery states, and the application must prevent invalid transitions.

1. Parcel Tracking States

The valid tracking sequence is:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED

Additional transitions:

- OUT_FOR_DELIVERY → FAILED_ATTEMPT
- FAILED_ATTEMPT → OUT_FOR_DELIVERY (maximum two retries)
- FAILED_ATTEMPT → RETURNED
- BOOKED → CANCELLED

Cancellation is allowed only before the parcel is picked up.

2. State-Transition Table

Current State	Next State	Validity
BOOKED	PICKED_UP	Valid
BOOKED	CANCELLED	Valid
PICKED_UP	IN_TRANSIT	Valid
IN_TRANSIT	OUT_FOR_DELIVERY	Valid
OUT_FOR_DELIVERY	DELIVERED	Valid
OUT_FOR_DELIVERY	FAILED_ATTEMPT	Valid
FAILED_ATTEMPT	OUT_FOR_DELIVERY	Valid, subject to retry limit
FAILED_ATTEMPT	RETURNED	Valid
PICKED_UP	CANCELLED	Invalid

IN_TRANSIT	CANCELLED	Invalid
BOOKED	DELIVERED	Invalid
DELIVERED	IN_TRANSIT	Invalid

3. Test Cases

Test ID	Current State	Requested Next State	Expected Result
ST01	BOOKED	PICKED_UP	Accepted
ST02	BOOKED	CANCELLED	Accepted
ST03	PICKED_UP	IN_TRANSIT	Accepted
ST04	IN_TRANSIT	OUT_FOR_DELIVERY	Accepted
ST05	OUT_FOR_DELIVERY	DELIVERED	Accepted
ST06	OUT_FOR_DELIVERY	FAILED_ATTEMPT	Accepted
ST07	FAILED_ATTEMPT	OUT_FOR_DELIVERY	Accepted within retry limit
ST08	FAILED_ATTEMPT	RETURNED	Accepted
ST09	PICKED_UP	CANCELLED	Rejected
ST10	BOOKED	DELIVERED	Rejected
ST11	DELIVERED	IN_TRANSIT	Rejected

4. Retry Limit Verification

The requirement permits a maximum of two retries. For testing, the interpretation should be confirmed with the product owner. Assuming two retries are permitted after the initial failed delivery attempt:

- First failed attempt: retry 1 is allowed.
- Second failed attempt: retry 2 is allowed.
- A further failed attempt must not permit another retry; the parcel should move to RETURNED.

The test suite must verify that the application does not allow more than two retries.

5. Importance of State-Transition Testing

This technique prevents parcels from skipping important delivery stages, being cancelled after pickup or moving backwards after delivery. It also ensures that retry limits are correctly enforced.

TASK 7: CONSTRUCT THE CONTROL FLOW GRAPH

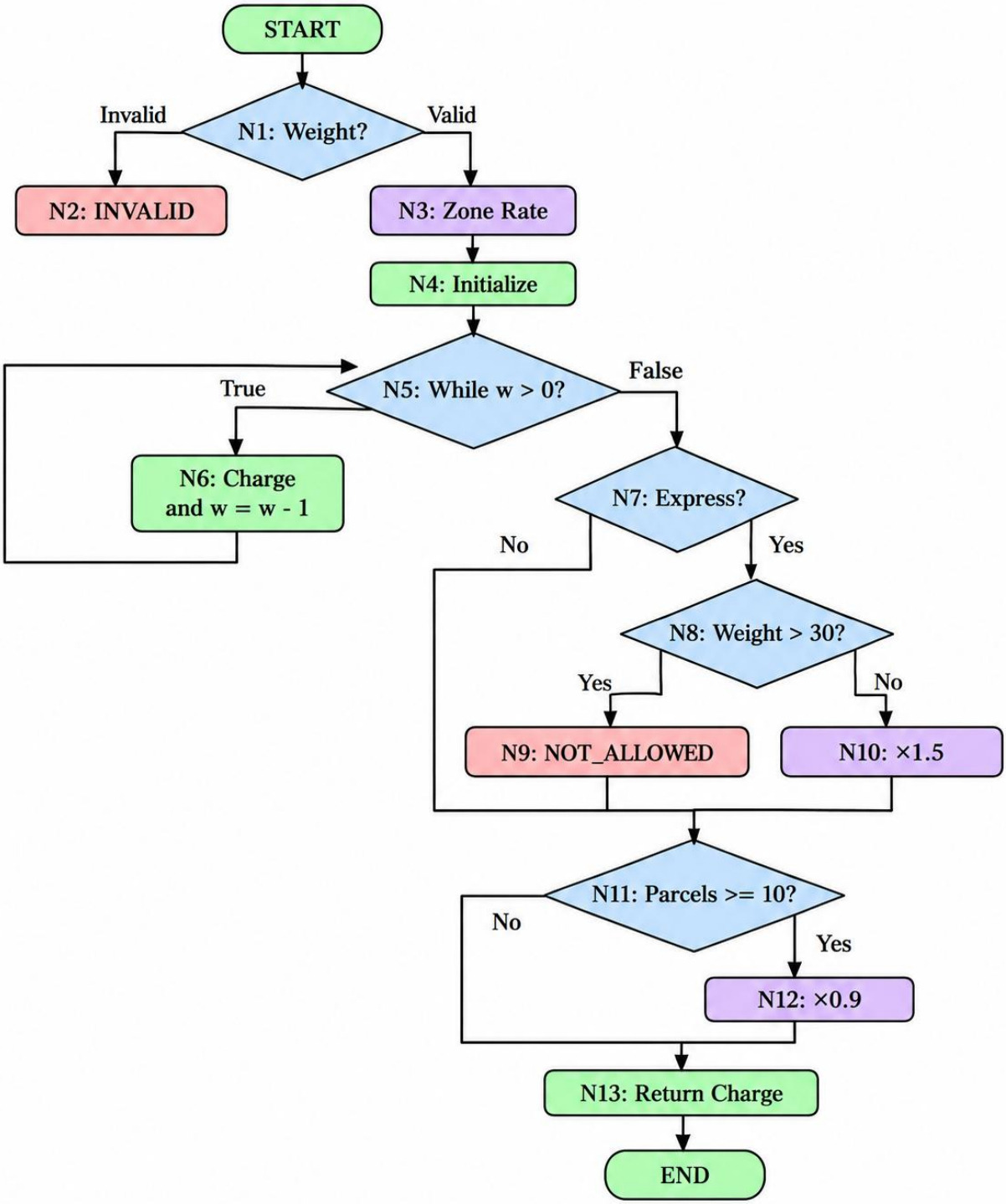
A Control Flow Graph (CFG) represents the execution flow of a program. It contains nodes representing statements or statement groups and directed edges representing the possible flow between them.

The SwiftShip pseudocode contains conditional statements, a loop, an express delivery decision and a discount decision.

1. Nodes and Their Corresponding Statements

Node	Pseudocode Lines	Description
N1	Start, line 1	Validate weight
N2	Line 2	Return INVALID
N3	Lines 3–7	Select delivery-zone rate
N4	Lines 8–9	Initialize charge and weight
N5	Line 10	Check loop condition
N6	Lines 11–12	Calculate charge and reduce weight
N7	Line 13	Check express option
N8	Line 14	Check express weight limit
N9	Line 15	Return NOT_ALLOWED
N10	Line 16	Apply express surcharge
N11	Line 17	Check discount eligibility
N12	Line 18	Apply discount
N13	Line 19	Return final charge

2. Control Flow Graph



3. Decision Nodes

The major decision points are:

1. Weight validation at line 1.
2. Zone B selection at line 4.
3. Zone C selection at line 6.
4. Weight loop condition at line 10.
5. Express option at line 13.
6. Express weight limit at line 14.
7. Discount condition at line 17.

4. Important Observation

The pseudocode does not contain a decision to validate whether the zone is A, B or C. Any other zone receives the default Zone A rate.

This is a mismatch between the specification and the implementation. The CFG represents the current pseudocode, not an ideal corrected implementation.

TASK 8: PATH TESTING AND CYCLOMATIC COMPLEXITY

Path Testing verifies different execution paths through a program. Cyclomatic Complexity measures the number of linearly independent paths in the control flow graph.

1. Cyclomatic Complexity Using Decision Nodes

Formula:

V(G) = Number of decision nodes + 1

The CFG contains seven decision nodes.

V(G) = 7 + 1

Cyclomatic Complexity = 8

The compound weight validation at line 1 is counted as one CFG decision node. Its two individual conditions can be examined separately during condition-coverage testing.

2. Cyclomatic Complexity Using the Edge-Node Formula

Formula:

$V(G) = E - N + 2P$

Where:

- E = Number of edges
- N = Number of nodes
- P = Number of connected components

For the CFG, the edge and node counts give the same complexity value of 8, assuming the nodes are grouped as in Task 7 and P = 1.

3. Independent Basis Paths

A basis path set contains independent execution paths. For a cyclomatic complexity of 8, eight independent paths are required.

Path	Execution Behaviour	Test Data
P1	Invalid weight returns INVALID	Weight = 0
P2	Zone A, normal delivery, no discount	1 kg, A, No, 9
P3	Zone B branch, normal delivery, no discount	1 kg, B, No, 9
P4	Zone C branch, normal delivery, discount applied	1 kg, C, No, 10
P5	Express enabled, eligible weight, no discount	1 kg, A, Yes, 9
P6	Express enabled, weight above 30 kg	31 kg, A, Yes, 9
P7	Normal delivery with discount	1 kg, A, No, 10
P8	Fractional weight causes multiple loop iterations	1.1 kg, A, No, 9

These paths exercise the important decisions and loop behaviour. The exact basis set can vary depending on how the CFG nodes and edges are represented.

4. Loop Testing

The loop calculates shipping charges for every started kilogram.

- One iteration: Weight = 1.0 kg.
- Multiple iterations: Weight = 3.0 kg.

- Fractional weight: Weight = 1.1 kg, which causes two iterations.
- Invalid weight: Weight = 0 returns before the loop.

For every valid weight, the loop executes at least once. A zero-iteration loop path is therefore not feasible for valid inputs under the current precondition.

5. Infeasible Paths

A path is infeasible when no possible input can execute it.

For example, the weight condition weight <= 0 OR weight > 50 cannot have both conditions true at the same time for a normal numerical input.

The tester should distinguish feasible paths from logically impossible paths when evaluating path coverage.

6. Evaluation

Cyclomatic Complexity helps estimate testing effort and identify the minimum number of independent paths. Path testing is useful for finding errors in decisions, loops and calculation sequences.

TASK 9: JUDGE TEST ADEQUACY AND COVERAGE

Test coverage measures how much of the software has been exercised by a test suite. Different coverage criteria provide different levels of confidence.

1. Statement Coverage

Statement coverage verifies that every executable statement is executed at least once.

For SwiftShip, tests should execute weight validation, rate selection, charge calculation, express processing, discount processing and return statements.

Formula:

Statement Coverage = (Executed Statements / Total Executable Statements) × 100

2. Branch Coverage

Branch coverage checks whether every decision has been evaluated as both true and false.

Examples include:

- Weight valid and invalid.
- Zone B condition true and false.
- Zone C condition true and false.
- Loop condition true and false.
- Express enabled and disabled.
- Express weight above and within 30 kg.
- Discount eligible and not eligible.

Formula:

Branch Coverage = (Executed Branches / Total Branches) × 100

3. Condition Coverage

Condition coverage verifies that each individual Boolean condition becomes both true and false.

For the weight condition:

weight <= 0 OR weight > 50

The following inputs can be used:

- Weight = 0: first condition true.
- Weight = 50.1: second condition true.
- Weight = 1: both conditions false.

4. Multiple-Condition Coverage

Multiple-condition coverage examines combinations of individual conditions within a compound decision.

For the weight validation condition, the relevant combinations are:

Weight	**weight <= 0**	**weight > 50**	**Overall Result**
0	True	False	INVALID
50.1	False	True	INVALID

1.0	False	False	Continue

Both conditions cannot be true simultaneously for the same ordinary numeric weight.

5. Additional Testing Required

To improve coverage, the following tests should be added:

- Invalid delivery zone such as X.
- Weight with more than one decimal place, such as 1.25 kg.
- Weight immediately below and above the express limit.
- Parcel counts 9 and 10.
- All valid tracking transitions.
- Invalid tracking transitions.
- Maximum retry-limit scenarios.

6. Can 100% Code Coverage Detect Every Defect?

No. Even 100% statement or branch coverage cannot guarantee that the application satisfies every requirement.

For example, the pseudocode has no invalid-zone validation branch. A test suite can execute every existing statement and branch while the application still accepts an invalid zone and calculates the charge using the Zone A rate.

Therefore, code coverage must be combined with requirement-based black-box testing.

Conclusion: Branch coverage, condition coverage and requirement-based testing together provide stronger evidence of software quality than statement coverage alone.

TASK 10: COMPARE BLACK-BOX AND WHITE-BOX TESTING EFFECTIVENESS

Black-box and white-box testing identify different categories of defects. Both techniques are necessary to evaluate the SwiftShip shipping-charge calculator.

1. Defects Identified Through Black-Box Testing

Black-box testing can identify:

- Invalid delivery zones being accepted.
- Weight values exceeding the permitted range.
- Weights containing more than one decimal place.
- Incorrect shipping charges.
- Incorrect express delivery restrictions.
- Incorrect discount application.
- Invalid parcel tracking transitions.
- Cancellation after pickup.

These defects are identified by comparing actual application behaviour with the functional requirements.

2. Defects Identified Through White-Box Testing

White-box testing can identify:

- Missing decision branches.
- Incorrect control flow.
- Loop execution errors.
- Incorrect calculation statements.
- Unreachable or redundant code.
- Uncovered execution paths.
- Errors in the internal implementation of conditions.

These defects are identified by examining the program structure and execution paths.

3. Defects That Can Be Identified by Both Techniques

Some defects can be identified using both approaches:

Defect	Black-Box Testing	White-Box Testing
Incorrect weight validation	Yes	Yes
Incorrect express restriction	Yes	Yes
Wrong discount calculation	Yes	Yes
Incorrect started-kilogram calculation	Yes	Yes
Missing invalid-zone validation	Yes	Can identify the missing decision
Invalid tracking transition	Yes	Yes, if transition logic is inspected

4. Why One Technique Is Not Enough

Black-box testing does not reveal the internal reason for a failure. It identifies incorrect external behaviour.

White-box testing may achieve high code coverage but still fail to detect missing business requirements that are not represented in the code.

For example, the current pseudocode does not implement invalid-zone validation. White-box testing of the existing branches alone may not identify that missing requirement unless the tester compares the implementation with the specification.

5. Recommended Approach

A combined testing strategy is recommended:

- Use ECP and BVA for input validation.
- Use Decision Table Testing for charge combinations.
- Use State-Transition Testing for parcel tracking.
- Use CFG and path testing for internal control flow.
- Use statement, branch and condition coverage to evaluate test adequacy.

This approach reduces duplicated tests while maintaining coverage of both functional requirements and internal logic.

TASK 11: RECOMMEND THE FINAL TEST SUITE

The final test suite should provide adequate functional and structural coverage while keeping the number of test cases and execution effort manageable.

1. Recommended Testing Techniques

Technique	Purpose
Equivalence Class Partitioning	Test valid and invalid input groups
Boundary Value Analysis	Test weight and discount limits
Decision Table Testing	Verify express and discount combinations
State-Transition Testing	Verify parcel tracking sequence
Path Testing	Execute independent program paths
Branch Coverage	Verify decision outcomes
Loop Testing	Verify started-kilogram calculations

2. Recommended Final Test Cases

Test ID	Input / Scenario	Expected Result	Technique
FT01	1 kg, Zone A, normal, 9 parcels	₹40	ECP
FT02	1.1 kg, Zone A, normal, 9 parcels	₹80	BVA
FT03	1 kg, Zone B, normal, 9 parcels	₹60	ECP
FT04	1 kg, Zone C, normal, 9 parcels	₹90	ECP
FT05	2 kg, Zone C, express, 9 parcels	₹270	Decision Table
FT06	1 kg, Zone A, express, 9 parcels	₹60	Decision Table
FT07	1 kg, Zone A, normal, 10 parcels	₹36	BVA
FT08	1 kg, Zone B, express, 10 parcels	₹81	Decision

			Table
FT09	30 kg, Zone A, express	₹1,800	BVA
FT10	30.1 kg, Zone A, express	NOT_ALLOWED	BVA
FT11	50 kg, Zone C, normal	₹4,500	BVA
FT12	50.1 kg, Zone A, normal	INVALID	BVA
FT13	0 kg, Zone A, normal	INVALID	BVA
FT14	Invalid zone X	INVALID	ECP
FT15	Weight 1.25 kg	INVALID	ECP
FT16	BOOKED → PICKED_UP	Accepted	State Transition
FT17	BOOKED → CANCELLED	Accepted	State Transition
FT18	PICKED_UP → CANCELLED	Rejected	State Transition
FT19	OUT_FOR_DELIVERY → FAILED_ATTEMPT	Accepted	State Transition
FT20	FAILED_ATTEMPT → OUT_FOR_DELIVERY within retry limit	Accepted	State Transition
FT21	Retry attempted beyond maximum limit	Rejected / RETURNED	State Transition
FT22	BOOKED → DELIVERED	Rejected	State Transition

3. Coverage Provided

The final suite covers:

- All three valid delivery zones.
- Valid and invalid weights.
- Minimum and maximum weight boundaries.

- Started-kilogram calculation.
- Express delivery eligibility.
- Discount eligibility.
- Invalid input values.
- Important tracking transitions.
- Retry limits and cancellation rules.
- Major independent execution paths.

4. Evaluation of the Final Suite

The suite combines different testing techniques instead of depending on one technique. This avoids unnecessary repetition while testing important business rules and internal program behaviour.

The test suite should be automated wherever possible, and failed test cases should be recorded with input values, actual output, expected output and defect details.

5. Remaining Risks

The pseudocode does not validate invalid zones or decimal precision. These defects must be corrected and retested. The requirements also do not specify validation rules for negative or fractional parcel counts or a currency-rounding rule.

Conclusion: The recommended suite provides a balanced combination of functional, boundary, decision, state-transition and structural testing. It is suitable as the regression test suite for future releases after the identified defects are corrected.

TASK 12: DAY-4 DESIGN CHALLENGE – ADAPTING THE TEST SUITE

SwiftShip introduces two additional requirements:

1. An 8% fuel surcharge for Zone C parcels.
2. A weekend delivery option that adds ₹50 per parcel.

These changes affect the shipping-charge calculation and require modifications to the existing test suite.

1. Impact of the New Requirements

The fuel surcharge introduces an additional decision based on the delivery zone. The weekend option introduces another decision based on whether weekend delivery is selected.

The updated calculation must consider:

- Weight-based charge.
- Delivery-zone rate.
- Express surcharge.
- Fuel surcharge for Zone C.
- Weekend delivery charge.
- Monthly parcel discount.

The existing test suite cannot be used without modification because the expected charges for some combinations have changed.

2. Evaluation of the Existing Test Suite

The existing suite already covers weight validation, delivery zones, express delivery, discount eligibility, tracking transitions and major program paths.

However, it does not cover fuel surcharge or weekend delivery.

Therefore, the existing suite should be retained as the foundation of the regression suite, with additional tests for the new requirements.

3. New Test Cases for Fuel Surcharge

Assuming the 8% surcharge is calculated on the shipping charge before the monthly discount:

Test ID	Zone	Base Charge	Fuel Surcharge	Expected Charge
DC01	A	₹40	₹0	₹40
DC02	B	₹60	₹0	₹60
DC03	C	₹90	₹7.20	₹97.20

The test cases verify that the surcharge is applied only to Zone C parcels.

4. New Test Cases for Weekend Delivery

Assuming the ₹50 weekend charge applies once to each parcel in the current shipment:

Test ID	Weekend Option	Base Charge	Additional Charge	Expected Charge
DC04	No	₹40	₹0	₹40
DC05	Yes	₹40	₹50	₹90

The weekend option must be tested both when selected and when not selected.

5. Combined Business Rule Testing

The new features must also be tested with existing express and discount rules.

For example, assuming the fuel surcharge is applied after the express multiplier and before the discount:

Weight = 1 kg
Zone = C
Express = Yes
Weekend = Yes
Monthly parcels = 10

Base charge = 1 × ₹90 = ₹90

Express charge = ₹90 × 1.5 = ₹135

Fuel surcharge = ₹135 × 8% = ₹10.80

Weekend charge = ₹50

Subtotal = ₹135 + ₹10.80 + ₹50 = ₹195.80

Discount = ₹195.80 × 10% = ₹19.58

Final charge = ₹195.80 - ₹19.58 = ₹176.22

This calculation is an example based on the stated assumptions. The assignment does not explicitly specify the order of the new charges or whether the weekend charge is discounted. These details must be confirmed before final expected results are approved.

6. Modified Decision Table

Conditions	R1	R2	R3	R4
Zone C?	Yes	No	Yes	No
Weekend selected?	No	No	Yes	Yes
Fuel surcharge	8%	0%	8%	0%
Weekend charge	₹0	₹0	₹50	₹50

This table verifies that the fuel surcharge depends on Zone C and the weekend charge depends on the weekend option.

7. Impact on Cyclomatic Complexity

The original pseudocode has a cyclomatic complexity of 8.

The two new requirements introduce two additional decisions:

- Fuel surcharge decision.
- Weekend delivery decision.

Assuming each introduces one additional decision node:

New Cyclomatic Complexity = 8 + 2

New Cyclomatic Complexity = 10

Therefore, the updated code requires additional independent path testing.

8. Regression Testing Strategy

The following tests should be included in the updated regression suite:

- Existing weight boundary tests.
- Existing zone A, B and C tests.
- New fuel surcharge tests for Zone C and non-Zone C.
- New weekend selected and not-selected tests.
- Combined express and fuel surcharge tests.
- Combined discount and weekend tests.

- Invalid weight and invalid zone tests.
- Existing parcel tracking and retry-limit tests.

Tests unrelated to the modified charge calculation, such as parcel tracking, should be retained because they protect existing functionality from regression defects.

9. Meeting the 30-Minute Execution Limit

The final regression suite must complete within 30 minutes before every release.

To achieve this:

1. Automate the repeated calculation and input-validation tests.
2. Prioritize high-risk financial calculation scenarios.
3. Execute boundary and decision-table tests for the new features.
4. Retain critical parcel tracking and retry-limit tests.
5. Remove only genuinely duplicate tests, not tests covering unique requirements.
6. Run the selected suite before each release and record execution time.
7. Execute broader exploratory and extended tests separately when needed.

A compact automated regression suite should be selected based on measured execution time. The actual number of tests that fit within 30 minutes must be confirmed through execution rather than assumed.

10. Requirement Clarifications

Before final approval, the following points must be clarified:

- Whether the fuel surcharge is calculated before or after the express surcharge.
- Whether the fuel surcharge is included in the amount eligible for the monthly discount.
- Whether the weekend charge is discounted.
- Whether ₹50 is added once per shipment or multiplied by the number of parcels in the shipment.
- Whether the existing parcels input represents monthly booking count only or also the number of parcels in the current shipment.

These clarifications are important because different calculation orders can produce different final charges.

Conclusion

The existing test suite should be retained and extended rather than completely redesigned. The new fuel surcharge and weekend delivery requirements introduce additional decisions and increase cyclomatic complexity from 8 to 10, assuming one decision for each feature.

A combination of Equivalence Class Partitioning, Boundary Value Analysis, Decision Table Testing, Path Testing and Regression Testing will provide effective coverage. The final automated regression suite must be validated against the 30-minute execution limit before every release.